webSPDZ · ATLAS — deferred-start & live-input prototype

De-risking the mechanism a future "lobby" UI will need: letting each of the 4 players type their own private input in the browser at runtime, instead of baking Player-Data into the WebAssembly build. Session date 2026-10-22, same machine/toolchain as webSPDZ-ATLAS-changes.pdf (emsdk 4.0.6, `-fwasm-exceptions`).

1. Why this was needed

Today, each player's 4 skill values live in `Player-Data/Input-Pn-0` on disk and get permanently embedded into `atlas-party.wasm` by `make -B atlas-party.x`. Changing a test case means editing files and rebuilding with the full Emscripten toolchain — not something you can hand to 4 people clicking through a web page. The goal: let a browser write its own player's input at the moment the page runs, with the value never leaving that browser.

Scope decided before starting: still a single-machine, 4-tab demo (not 4 separate computers), and input values must never be sent to any server — only readiness/coordination signals would ever leave the browser in the full lobby design. This prototype only covers the lower mechanism; there is still no lobby UI — parties are started by hand from the DevTools console.

2. What changed

File	Change
`Makefile` (WEB=1 LDFLAGS, line ~166)	Added `-sEXPORTED_RUNTIME_METHODS=FS,callMain` so the build actually exposes `Module.FS` and `Module.callMain` to page JavaScript — neither was reachable from outside before.
`webspdz-shell.html`	Added `noInitialRun: true` to the `Module` object literal, so the generated runtime does setup (memory, filesystem, WASM link) but does **not** call `main()` on its own.
`local/testing-post.js`	Rewritten to support two start modes from one build (see §3) — the legacy `?arguments=...` URL flow, and a new `window.webSPDZStartParty(args, inputFiles)` hook.

`atlas-party.html`, `atlas-party.js` are regenerated build outputs (rebuilt via `make -B atlas-party.x -j2`) — not hand-edited. `atlas-party.wasm` came out byte-identical to before; none of this touched the compiled C++ logic.

Shared files, scoped rebuild. `local/testing-post.js` and `webspdz-shell.html` are used by every `*-party.x` web target (Shamir, Rep4-Ring, Semi2k, etc.), not just ATLAS. Only `atlas-party.x` was rebuilt this session, so only its `.html` / `.js` picked up these changes — the other already-built protocol pages are untouched until someone rebuilds them too.

3. How the dual-mode start works

```
arguments_ = [];
(function () {
  var runtimeReady = false, pending = null;

  function doStart(args, inputFiles) {
    Object.keys(inputFiles || {}).forEach(function (path) {
      var slash = path.lastIndexOf('/');
      if (slash > 0) Module['FS'].mkdirTree(path.substring(0, slash));
      try { Module['FS'].truncate(path, 0); } catch (e) {}   // see Problem 3
      Module['FS'].writeFile(path, inputFiles[path]);
    });
    arguments_ = args;
    Module['callMain'](args);
  }

  Module['onRuntimeInitialized'] = function () {              // see Problem 2
    runtimeReady = true;
    if (pending) { doStart(pending.args, pending.inputFiles); pending = null; }
```

```
  };

  window.webSPDZStartParty = function (args, inputFiles) {
    runtimeReady ? doStart(args, inputFiles) : (pending = { args, inputFiles });
  };

  var sp = new URLSearchParams(window.location.search);
  if (sp.has('arguments')) window.webSPDZStartParty(sp.get('arguments').split(','));
})();
```

Both modes end up calling the same `doStart()` — the only difference is where `args` / `inputFiles` come from: the URL query string (legacy), or a direct JS call (new).

4. Problems hit, in order, and how they were resolved

PROBLEM 1

Edits to `atlas-party.html` silently disappeared

What happened: first attempt edited `Module={...}` directly inside `atlas-party.html` to add `noInitialRun:true`. The edit was there — until the next `make -B atlas-party.x`, after which it was gone.

Cause: `atlas-party.html` is a *build output*, not a source file. The link step runs with `--shell-file webspdz-shell.html`, which regenerates `atlas-party.html` from that template on every build — any direct edit to the generated file is discarded on the next rebuild.

Fix: make the same edit in `webspdz-shell.html` instead, then rebuild.

PROBLEM 2

Calling `Module.callMain()` crashed every tab

```
TypeError: Cannot read properties of undefined (reading '_a')
  at __emscripten_stack_alloc
  at stackAlloc
  at Object.callMain
```

Cause: with `noInitialRun` set, the normal auto-run path never calls `callMain` — so the first version of `testing-post.js` called `Module['callMain'](args)` itself, synchronously, right at script-parse time. But the actual `.wasm` file is still being fetched and linked asynchronously at that point — its exports object doesn't exist yet. Calling into WASM before it's linked crashes.

Fix: never call `callMain` directly from the top-level script. Instead hook `Module['onRuntimeInitialized']` — the callback Emscripten's own dependency-gated `run()` / `doRun()` fires exactly once the WASM module is linked and ready — and only call `callMain` from there (immediately if already ready, or once it fires, via a small `pending` queue).

PROBLEM 3

Injected input was silently ignored — `FS.writeFile` appends instead of overwriting

What happened: after fixing Problem 2, all 4 parties ran and produced a result — but it was the exact same result as the old baked-in demo values, even though fresh numbers had just been written in. No crash, no error — just stale data winning.

Diagnosis: read the file straight back after writing it:

```
FS.writeFile('Player-Data/Input-P0-0', '5\n5\n5\n5\n');
FS.readFile('Player-Data/Input-P0-0', {encoding:'utf8'});
// -> "4\n13\n6\n10\n5\n5\n5\n5\n"   (old content, THEN the new content appended)
```

Tried `{flags:'w'}`, tried `FS.unlink()` before writing — same result both times. A separate probe explained why: `FS.stat()` on the same path throws `TypeError: Cannot convert 2174336 to a BigInt` — a real toolchain bug in this exact combination (WASMFS backend + `-sMEMORY64` + the exported `FS` runtime method in Emscripten 4.0.6): file-size bookkeeping that should return a 64-bit size silently breaks, and `writeFile` falls back to writing at end-of-file instead of truncating.

Fix: sidestep `writeFile`'s broken truncate path entirely — call the lower-level `FS.truncate(path, 0)` explicitly before every write (wrapped in `try/catch` so it's a no-op on a path that doesn't exist yet). Verified directly:

```
FS.truncate('Player-Data/Input-P0-0', 0);
FS.readFile(...) // -> ""              (confirmed empty)
FS.writeFile('Player-Data/Input-P0-0', '5\n5\n5\n5\n');
FS.readFile(...) // -> "5\n5\n5\n5\n"   (exactly the new content, nothing stale)
```

5. How it was verified (not just "didn't crash")

Automated with Playwright driving the real, installed Chrome (`channel: 'chrome'`, no separate browser download) — 4 tabs in one script, console output captured per tab.

- **Regression check:** the 4 existing `?arguments=-N,4,-T,1,-w,0,<i>,surplus_matching_private,-OF,.` URLs (same ones the Docker demo and setup guides use) still produce the same result as before any of this — **3, 2, 1, 0**. Confirms nothing already working got broken.
- **New capability, correctness-checked:** 4 tabs loaded with no URL parameters at all, each driven by `webSPDZStartParty` with inputs chosen specifically to hand-verify against the matching algorithm in `Programs/Source/surplus_matching_private.mpc` (T=10 inside the program):

Party	Input (4 skills)	Profile
P0	`5, 5, 5, 5`	pure deficit — needs help everywhere
P1	`15, 15, 15, 15`	pure surplus — can help everywhere
P2	`10, 10, 10, 10`	balanced — nothing to give or receive
P3	`10, 10, 10, 10`	balanced — nothing to give or receive

  By hand: only the (P0, P1) pair has anything useful to trade, so matching `M0 = (0,1)+(2,3)` wins outright, giving **P0→1, P1→0, P2→-1, P3→-1**. The automated run returned exactly that. Since this differs from the old baked-in result, and matches an independent hand calculation, it proves the injected values were genuinely used — not a lucky pass-through of stale data.

Net result: deferred `main()` + runtime `FS` injection is viable on this exact build (WASMFS + PROXY_TO_PTHREAD + MEMORY64), with the two fixes above. Safe to build the lobby UI on top of `window.webSPDZStartParty`.

6. Exact instructions to try it

Servers already running from the earlier setup — nothing new to start:

```
python3 https-server.py localhost 8000      # terminal A
node signaling_wss_server.js localhost 8080 # terminal B
```

1. Open 4 Chrome tabs, all to the same plain URL — **no `?arguments=`**:

   ```
   https://localhost:8000/atlas-party.html
   ```

2. Open DevTools (F12) → Console in each tab.
3. Paste one block per tab (values match the §5 test — feel free to substitute your own 4 numbers per party; T=10, so above 10 is surplus, below is deficit):

Tab 1 — P0

```
window.webSPDZStartParty(
  ['-N','4','-T','1','-w','0','0','surplus_matching_private','-OF','.'],
  {'Player-Data/Input-P0-0': '5\n5\n5\n5\n'}
);
```

Tab 2 — P1

```
window.webSPDZStartParty(
  ['-N','4','-T','1','-w','0','1','surplus_matching_private','-OF','.'],
  {'Player-Data/Input-P1-0': '15\n15\n15\n15\n'}
);
```

Tab 3 — P2

```
window.webSPDZStartParty(
  ['-N','4','-T','1','-w','0','2','surplus_matching_private','-OF','.'],
  {'Player-Data/Input-P2-0': '10\n10\n10\n10\n'}
);
```

Tab 4 — P3

```
window.webSPDZStartParty(
  ['-N','4','-T','1','-w','0','3','surplus_matching_private','-OF','.'],
  {'Player-Data/Input-P3-0': '10\n10\n10\n10\n'}
);
```

Run all 4 within a few seconds of each other — they wait for each other to connect, same as the URL-based flow. Expect **P0 → 1, P1 → 0, P2 → -1, P3 → -1**.

The old URL-based flow still works unchanged, for comparison:

```
https://localhost:8000/atlas-party.html?arguments=-N,4,-T,1,-w,0,0,surplus_matching_private,-OF,.
https://localhost:8000/atlas-party.html?arguments=-N,4,-T,1,-w,0,1,surplus_matching_private,-OF,.
https://localhost:8000/atlas-party.html?arguments=-N,4,-T,1,-w,0,2,surplus_matching_private,-OF,.
https://localhost:8000/atlas-party.html?arguments=-N,4,-T,1,-w,0,3,surplus_matching_private,-OF,.
```

7. What this is not yet

- No lobby UI — starting a party means pasting JS into DevTools, by design, to isolate and prove the underlying mechanism first.
- No party-index assignment or room coordination — each tab still has to be told its own index (0–3) by hand.
- Docker demo image is unaffected (still serves the legacy baked-in-input build) and doesn't need to change for this.

Next step: build the lobby — a page all 4 players open identically, that assigns party slots over the existing signaling WebSocket server, collects each player's input through a form, and calls `webSPDZStartParty` automatically once all 4 are ready. Input values still never leave the browser they were typed into.
